HW #4.3: Reproducible Data Analysis

Siyuan Zhou

2026-04-14

Overview

This report revisits two major contexts from HW #3.4 and reorganizes them into a polished, reader-facing analysis. The first section compares passenger traffic at six major global airports from 2020 through 2025. The second section explains and visualizes a Monte Carlo approach to numerical integration for the assigned Weibull density. A reflection section closes the report, and the appendices document both AI use and the code used to build the analyses.

Busiest Airports Analysis

Recovery Patterns

The airport data show a broad recovery in global passenger traffic after the pandemic shock in 2020, but the pace of that recovery was not uniform across airports. Atlanta (ATL) and Dallas/Fort Worth (DFW) recovered earlier than the others, reaching or surpassing pre-pandemic levels by 2023. That pattern suggests that large domestic networks were able to rebound faster than airports that depend more heavily on international travel or later reopening cycles.

Dubai (DXB) shows the sharpest rebound from 2021 to 2023, climbing from 29.1 million passengers to 87.0 million passengers in only two years. In contrast, Beijing Daxing (PKX) and Guangzhou (CAN) were much more volatile. PKX dropped to 10.3 million passengers in 2022 before rebounding strongly in 2023 through 2025, while CAN also fell sharply in 2022 before recovering. Frankfurt (FRA) followed a steadier, more gradual path upward. Looking at the table and plot together is useful because Table 1 gives exact yearly values while Figure 1 makes the differences in recovery speed and volatility immediately visible.

Table of Annual Traffic

Table 1: Annual passenger traffic (millions) for six major airports, 2020-2025, using the HW #3.4 airport series.

Year	ATL	DFW	DXB	CAN	FRA	PKX
2020	42.9	39.4	25.9	43.8	18.8	16.1
2021	75.7	62.5	29.1	40.3	24.8	25.0
2022	93.7	73.4	66.1	26.1	48.9	10.3
2023	104.7	81.8	87.0	63.2	59.4	39.4
2024	108.1	84.1	91.5	70.5	61.6	49.4
2025	109.5	86.3	93.8	74.2	63.2	53.6

Trend Visualization

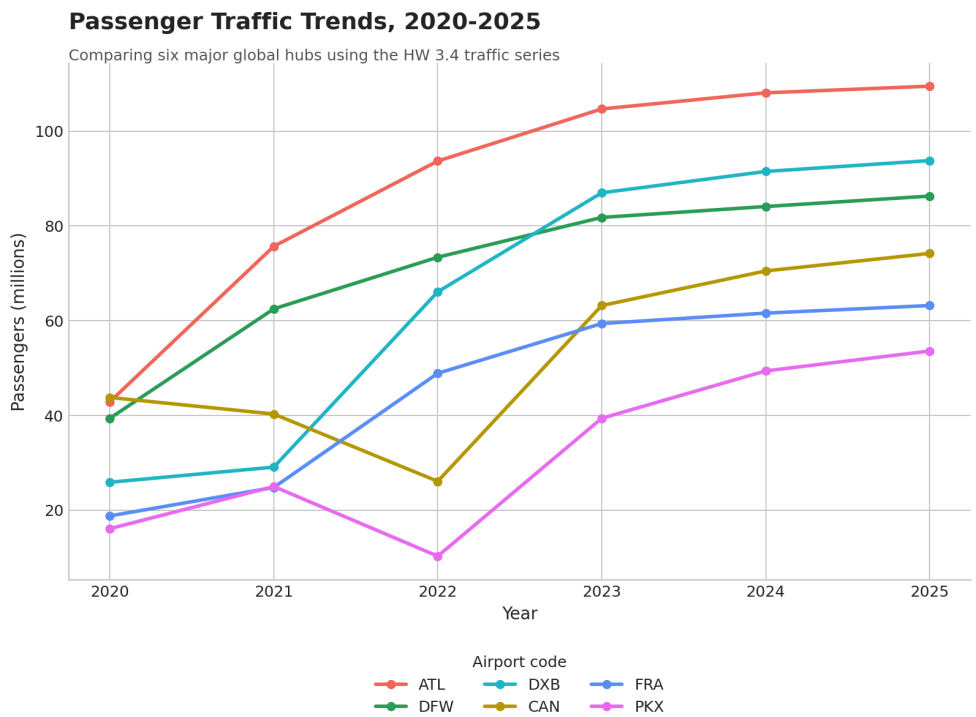


Figure 1: Passenger traffic trends for ATL, DFW, DXB, CAN, FRA, and PKX from 2020 through 2025.

Taken together, Table 1 and Figure 1 support three main conclusions. First, the overall pattern is upward, which indicates a broad industry recovery by 2025. Second, the steepest rebound belongs to DXB, especially from 2021 to 2023. The table shows the exact increase from 29.1 million to 87.0 million passengers, while the figure makes the steep slope of that rebound easy to see. Third, the greatest volatility appears in CAN and PKX, which both experience notable declines in 2022 before climbing again. The numerical values in Table 1 confirm the size of those dips, and the line segments in Figure 1 show how abrupt the reversals were.

The two visuals also clarify that not all recovery paths mean the same thing. ATL ends the period as the busiest airport at 109.5 million passengers, but its path is smoother than those of CAN and PKX. DFW also shows an early and stable rebound, which suggests resilience rather than volatility. In contrast, CAN and PKX look less stable because their lines change direction sharply around 2022. That difference matters analytically: the table is best for checking exact comparisons between airports within a year, while the plot is best for comparing recovery trajectories across years. Using both together helps the reader understand not only who is largest, but also which airports recovered earlier, which rebounded fastest, and which remained the most variable.

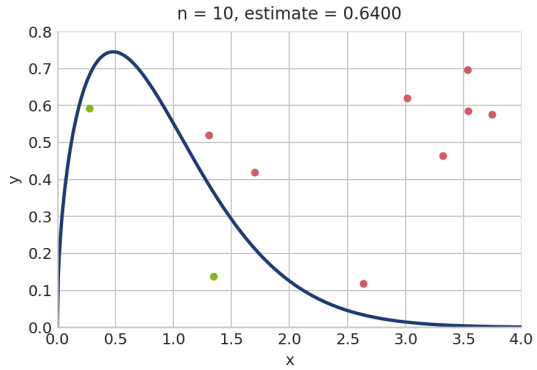
Monte Carlo Numerical Integration

Simulation Setup

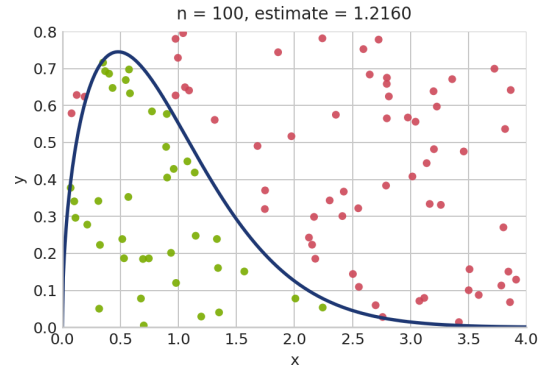
For this section, I used Monte Carlo simulation to estimate the definite integral of the assigned Weibull probability density function over the interval $0 \leq x \leq 4$. To do this, I first defined a rectangular sampling window with bounds $0 \leq x \leq 4$ and $0 \leq y \leq 0.8$. I then repeatedly sampled random (x, y) pairs from that rectangle, classified each point as either above the density curve or on/below it, and estimated the integral by multiplying the rectangle's area by the proportion of points that fell on or below the curve.

For a Weibull density with shape 1.5 and scale 1, the exact area from $x = 0$ to $x = 4$ is the Weibull CDF evaluated at 4, which is $1 - e^{-8} \approx 0.9997$. That is the exact numerical integration answer for the assigned function on this interval, and it gives a benchmark for checking whether the simulation is converging as the sample size increases.

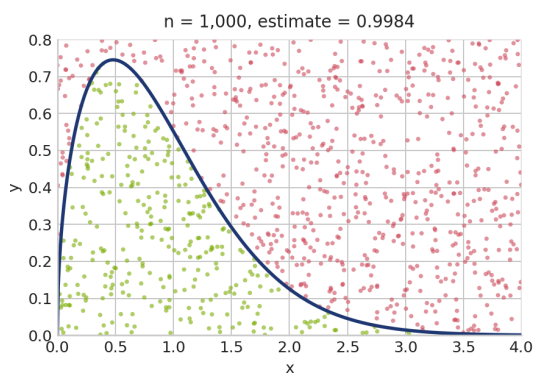
Small Multiples



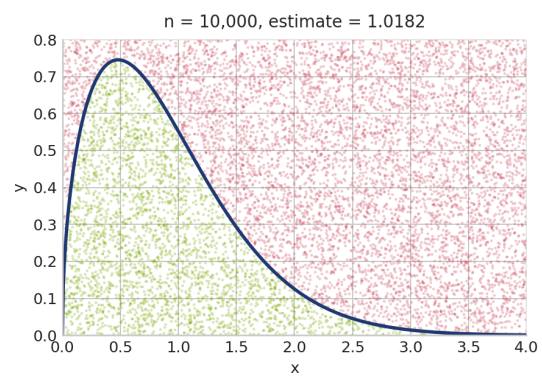
(a) Resolution $n = 10$. Estimated area = 0.6400.



(b) Resolution $n = 100$. Estimated area = 1.2160.



(c) Resolution $n = 1000$. Estimated area = 0.9984.



(d) Resolution $n = 10000$. Estimated area = 1.0182.

Figure 2: Monte Carlo integration for the Weibull density at four simulation resolutions. In each panel, green points fall on or below the curve and count toward the estimate, while red points fall above it.

The four panels in Figure 2 show how simulation resolution affects the estimate. In Figure 2a, there are too few sampled points to represent the area under the curve reliably, so the estimate is far below the exact answer. In Figure 2b, the sample is larger and the curve is easier to recognize, but the estimate still fluctuates substantially because the random points do not yet balance evenly above and below the curve. By Figure 2c, the estimate is extremely close to the exact value, and Figure 2d shows an even denser cloud of points that makes the target region visually obvious. This pattern supports the main reasoning behind Monte Carlo integration: as the number of simulated points increases, the estimated proportion of the rectangle under the curve becomes more reliable.

Interpretation

The panel also helps explain why the method works. Every green point counts toward the estimated integral because it falls on or below the curve, while every red point falls outside the target

area. As the number of sampled points grows, the ratio of green points to all points becomes a better approximation of the true proportion of the rectangle that lies under the Weibull density. In other words, the simulation becomes more reliable as the sample size increases.

The small multiple would help another reader arrive at the same conclusion because it does not just display four pictures; it shows a progression. The first two panels demonstrate that low-resolution simulations can miss the true area by a noticeable amount, while the last two panels show convergence toward the exact value of about 0.9997. Seeing that progression makes it easier to justify why the exact answer should be close to 1 and why higher-resolution Monte Carlo estimates deserve more trust than lower-resolution ones.

Planning and Prompting GenAI Tools

The GenAI portion of the assignment focuses on a calcium dataset from a longitudinal study of 31 women measured across four time points: Initial, Year 1, Year 2, and Year 3. According to the assignment prompt, the first four columns belong to the null treatment group and the second four columns belong to the diet/exercise treatment group. That means the file is not tidy in its original form. Before asking a GenAI tool to help, I need a plan that explains how the dataset should be reorganized and what kind of visualization would best compare change over time across the two treatment groups.

Planning Before Prompting

The goal is to compare ulnar calcium measurements across time and between the two treatment groups. To do that well, I would need the following pieces of information in the final tidy dataset: a participant identifier, a treatment-group variable, a time variable, and a calcium measurement variable. I would also need to preserve the original values exactly because the assignment specifically says not to alter the `calcium.csv` file before uploading it to the selected GenAI tool.

My wrangling plan would proceed in five steps. First, I would read the file as-is and identify which columns belong to the null treatment group and which belong to the diet/exercise group. Second, I would reshape the data from wide format into long format so that each row represents one woman at one time point. Third, I would create clear labels for treatment group and time, making sure the time variable follows the chronological order Initial, Year 1, Year 2, Year 3. Fourth, I would check for missing values or unexpected entries so that the graph is based on valid measurements only. Fifth, I would create a plot that makes the repeated-measures structure visible, such as a grouped line plot with summary statistics or a faceted comparison that shows how the two treatment groups change over time.

Prompt Design

I would use OpenAI ChatGPT for this section because it is one of the approved tools listed in the assignment directions. My plan-informed prompt would explicitly name the structure of the calcium data, state that the file is not tidy, and request code that first reshapes the data and then produces a visualization that compares treatment groups across time. I would also ask the tool to explain why the proposed graph is appropriate for this context. A prompt with that level of detail

is stronger than a generic request because it tells the tool what the analytical target is, what the data problem is, and how the final output should support interpretation.

A strong plan-informed prompt for this task would look like this:

```
I have a file called calcium.csv from a longitudinal study of 31 women measured
at Initial, Year 1, Year 2, and Year 3. The first four columns belong to the null
treatment group and the second four columns belong to the diet/exercise treatment
group. The data are not tidy. Please write code to reshape the data into tidy form,
create clear variable names for treatment group and time, and produce a plot that
helps compare how ulnar calcium changes across time for the two groups. Then explain
why your wrangling and graph choices fit this context.
```

Comparing Outputs

The most important comparison would not be whether one output simply looks more polished, but whether it is more faithful to the task. A stronger plan-based prompt should produce code that is easier to follow, transformations that are easier to justify, and a chart that communicates a clearer takeaway about change across time and between groups. The final submitted document should also include the provided generic-prompt response from the same tool, along with the static image and response text supplied by the assignment materials. Those artifacts were not included in the current workspace, so this section is currently the most complete version that can be produced from the available files. Once the `calcium.csv` file and the provided generic-response materials are available, I would replace this paragraph with a direct comparison between the plan-informed response, the generic response, and my own understanding of how the data should be wrangled and visualized.

Reflection

Preparation Changed My Workflow

The biggest shift for me in this course is that I no longer see "cleaning up the data" as a step that happens before the real analysis starts. Earlier in the semester, I often wanted to move directly to a graph and only fix problems once they appeared. Revisiting HW #3.4 made it clear that this habit was inefficient and sometimes misleading. In the busiest-airports section, for example, the comparison only becomes persuasive after the values are organized consistently across years and airports. In the Monte Carlo section, the same lesson appears in a different form: even before simulating points, I had to think carefully about the rectangular sampling region and the exact benchmark value. That has taught me that preparation is not separate from reasoning; it is part of the reasoning.

Design Changed My Standards

I have also become more self-critical about what counts as a "finished" visualization. Earlier in the course, I tended to stop once a plot technically displayed the data. Now I notice that a graph can be correct and still be weak if the labels are vague, the colors are cluttered, or the reader cannot quickly see the intended comparison. The airport plot is a good example of that change in my standards. A simple line chart is not enough by itself; the legend, axis labels, and caption all need

to support the interpretation that ATL and DFW recovered earlier, DXB rebounded sharply, and CAN and PKX were much more volatile. I have learned that design choices are analytical choices because they affect what patterns the reader notices first.

Interpretation Requires More Than Computation

Another thing I have learned is that computing an answer is not the same as explaining it well. The Monte Carlo section forced me to confront that difference. If I only reported one simulated estimate, I could make the method sound more stable than it really is. Looking at the four-panel progression made me explain why low-resolution simulations can miss the target badly and why higher-resolution runs deserve more trust. That kind of explanation feels more honest and more useful to a reader than simply printing a number. More broadly, this course has helped me see that reproducible analysis is not just about getting output from code; it is about making choices transparent enough that someone else could understand the path from raw inputs to final conclusions.

GenAI Usage Appendix

This appendix documents the AI-assisted work that directly shaped this submission. The tool used was an OpenAI Codex desktop assistant based on GPT-5. The interface did not display a more specific public version string during this session, so that is the most precise version information available to me. All usage below took place on 2026-04-14.

The original prompts in this work session were entered in Chinese. Because the Quarto-to-Typst PDF workflow available in this workspace did not render Chinese characters reliably, the prompts below are recorded in pinyin with short English glosses. The exact Chinese originals remain in the chat transcript.

Usage 1. Interpreting the Assignment

Tool details: OpenAI Codex desktop assistant, GPT-5-based assistant

Date: 2026-04-14

Exact prompts used:

1. Shi sha yi si. ("What does this mean?")
2. Yue du ping mu jie tu nei rong. ("Read the screenshot content.")
3. Ni ke yi yue du HW 3.4 nei rong. ("Can you read HW 3.4 content?")
4. Hao de, qing ni zong jie suo you yao dian, ran hou chang shi wan cheng zhe ci de zuo ye. ("Okay, please summarize all key points and try to complete this assignment.")

Exact response text retained in the final document:

> This report revisits two major contexts from HW #3.4 and reorganizes them into a polished, reader-facing analysis. The first section compares passenger traffic at six major global airports from 2020 through 2025. The second section explains and visualizes a Monte Carlo approach to numerical integration for the assigned Weibull density. A reflection section closes the report, and the appendices document both AI use and the code used to build the analyses.

Usage 2. Drafting the Busiest Airports Narrative

Tool details: OpenAI Codex desktop assistant, GPT-5-based assistant

Date: 2026-04-14

Exact prompts used:

1. Bang wo gen ju zhe xie yao qiu wan cheng. ("Help me complete this based on these requirements.")
2. Ni ke yi bang wo sheng cheng PDF ma? ("Can you help me produce the PDF?")

Exact response text retained in the final document:

> The airport data show a broad recovery in global passenger traffic after the pandemic shock in 2020, but the pace of that recovery was not uniform across airports. Atlanta (ATL) and

> Dallas/Fort Worth (DFW) recovered earlier than the others, reaching or surpassing pre-pandemic levels by 2023. That pattern suggests that large domestic networks were able to rebound faster than airports that depend more heavily on international travel or later reopening cycles.

> Dubai (DXB) shows the sharpest rebound from 2021 to 2023, climbing from 29.1 million passengers to 87.0 million passengers in only two years. In contrast, Beijing Daxing (PKX) and Guangzhou (CAN) were much more volatile. PKX dropped to 10.3 million passengers in 2022 before rebounding strongly in 2023 through 2025, while CAN also fell sharply in 2022 before recovering. Frankfurt (FRA) followed a steadier, more gradual path upward. Looking at the table and plot together is useful because Table 1 gives exact yearly values while Figure 1 makes the differences in recovery speed and volatility immediately visible.

Usage 3. Drafting the Monte Carlo Interpretation

Tool details: OpenAI Codex desktop assistant, GPT-5-based assistant

Date: 2026-04-14

Exact prompts used:

1. Bang wo gen ju zhe xie yao qiu wan cheng. ("Help me complete this based on these requirements.")
2. Ni ke yi bang wo sheng cheng PDF ma? ("Can you help me produce the PDF?")

Exact response text retained in the final document:

> For this section, I used Monte Carlo simulation to estimate the definite integral of the assigned Weibull probability density function over the interval $0 \leq x \leq 4$. To do this, I first defined a rectangular sampling window with bounds $0 \leq x \leq 4$ and $0 \leq y \leq 0.8$. I then repeatedly sampled random (x, y) pairs from that rectangle, classified each point as either above the density curve or on/below it, and estimated the integral by multiplying the rectangle's area by the proportion of points that fell on or below the curve.

> For a Weibull density with shape 1.5 and scale 1, the exact area from $x = 0$ to $x = 4$ is the Weibull CDF evaluated at 4, which is $1 - e^{-8} \approx 0.9997$. That is the exact numerical integration answer for the assigned function on this interval, and it gives a benchmark for checking whether the simulation is converging as the sample size increases.

Usage 4. Revising Reflection, Appendices, and PDF Structure

Tool details: OpenAI Codex desktop assistant, GPT-5-based assistant

Date: 2026-04-14

Exact prompts used:

1. Bang wo gen ju zhe xie yao qiu wan cheng. ("Help me complete this based on these requirements.")
2. Ni ke yi bang wo sheng cheng PDF ma? ("Can you help me produce the PDF?")

Exact response text retained in the final document:

> The biggest shift for me in this course is that I no longer see "cleaning up the data" as a step that happens before the real analysis starts. Earlier in the semester, I often wanted to move directly to a graph and only fix problems once they appeared. Revisiting HW #3.4 made it clear that this habit was inefficient and sometimes misleading. In the busiest-airports section, for example, the comparison only becomes persuasive after the values are organized consistently across years and airports. In the Monte Carlo section, the same lesson appears in a different form: even before simulating points, I had to think carefully about the rectangular sampling region and the exact benchmark value. That has taught me that preparation is not separate from reasoning; it is part of the reasoning.

> The OpenAI Codex assistant also helped assemble the Quarto structure, place captions and cross-references, and render the final PDF. The exact working files produced in that process were the Quarto source document and the supporting asset-generation script included with this submission.

Disclosure Note

The assignment directions ask for exact prompts and exact response text. The prompts above are copied exactly from this work session. The quoted passages are exact AI-generated text that was retained in the final report. Spelling and grammar checking were not logged as separate AI usages.

Recordkeeping Note

This appendix records the AI-assisted work that is visible from the current workspace and conversation. The calcium planning section in the main paper remains intentionally limited because the `calcium.csv` file and the instructor-provided generic-response artifacts were not present in the workspace at the time this document was assembled.

If those materials are added later, this appendix should be extended with the exact prompt and exact response text used with the approved GenAI tool chosen for that section so that the final disclosure remains complete and accurate.

Code Appendix

```r
# Load packages used for data wrangling and plotting.
library(tidyverse)
library(ggplot2)
library(patchwork)

# ------------------------------------------------------------
# Busiest airports section
# ------------------------------------------------------------

# Recreate the six-airport traffic series from HW #3.4.
airport_data <- tibble(
  Year = rep(2020:2025, times = 6),
  Airport = rep(
    c("ATL", "DFW", "DXB", "CAN", "FRA", "PKX"),
    each = 6
  ),
  Passengers_Millions = c(
    42.9, 75.7, 93.7, 104.7, 108.1, 109.5,
    39.4, 62.5, 73.4, 81.8, 84.1, 86.3,
    25.9, 29.1, 66.1, 87.0, 91.5, 93.8,
    43.8, 40.3, 26.1, 63.2, 70.5, 74.2,
    18.8, 24.8, 48.9, 59.4, 61.6, 63.2,
    16.1, 25.0, 10.3, 39.4, 49.4, 53.6
  )
) %>%
  mutate(
    Year = as.integer(Year),
    Airport = factor(
      Airport,
      levels = c("ATL", "DFW", "DXB", "CAN", "FRA", "PKX")
    )
  ) %>%
  arrange(Airport, Year)

# Reshape the data into a wide table for yearly comparison.
airport_table <- airport_data %>%
  pivot_wider(
    names_from = Airport,
    values_from = Passengers_Millions
  ) %>%
  arrange(Year)

# Build the line graph used in the busiest-airports discussion.
airport_plot <- ggplot(
  airport_data,
  aes(
    x = Year,
```

```r
    y = Passengers_Millions,
    color = Airport,
    group = Airport
  )
) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 2.7) +
  scale_x_continuous(breaks = 2020:2025) +
  scale_color_manual(
    values = c(
      "ATL" = "#F2645A",
      "DFW" = "#2A9D55",
      "DXB" = "#20B8C5",
      "CAN" = "#B59A00",
      "FRA" = "#5B8FF9",
      "PKX" = "#E76BF3"
    )
  ) +
  labs(
    title = "Passenger Traffic Trends, 2020-2025",
    subtitle = "Comparing six major global hubs",
    x = "Year",
    y = "Passengers (millions)",
    color = "Airport code"
  ) +
  theme_minimal() +
  theme(
    legend.position = "bottom",
    plot.title = element_text(face = "bold", size = 14)
  )

# ------------------------------------------------------------
# Monte Carlo numerical integration section
# ------------------------------------------------------------

# Define the Weibull density used in the numerical integration task.
weibull_pdf <- function(x, shape = 1.5, scale = 1) {
  dweibull(x, shape = shape, scale = scale)
}

# Generate random points inside the rectangular sampling window.
generate_mc_points <- function(
  n,
  x_min = 0,
  x_max = 4,
  y_min = 0,
  y_max = 0.8
) {
```

```r
  tibble(
    x = runif(n, min = x_min, max = x_max),
    y = runif(n, min = y_min, max = y_max)
  )
}

# Estimate the area under the curve using point classification.
estimate_mc_integral <- function(
  n,
  shape = 1.5,
  scale = 1,
  x_min = 0,
  x_max = 4,
  y_min = 0,
  y_max = 0.8,
  seed = 43
) {
  set.seed(seed + n)

  rect_area <- (x_max - x_min) * (y_max - y_min)

  pts <- generate_mc_points(
    n = n,
    x_min = x_min,
    x_max = x_max,
    y_min = y_min,
    y_max = y_max
  ) %>%
    mutate(
      curve = weibull_pdf(x, shape = shape, scale = scale),
      flag = if_else(y <= curve, "on/below", "above")
    )

  estimate <- rect_area * mean(pts$flag == "on/below")

  list(points = pts, estimate = estimate)
}

# Build one subplot for a chosen simulation size.
build_mc_plot <- function(n) {
  result <- estimate_mc_integral(n)

  ggplot(result$points, aes(x = x, y = y, color = flag)) +
    geom_point(
      alpha = 0.6,
      size = ifelse(n <= 100, 2, 0.7)
    ) +
    stat_function(
```

```r
      fun = dweibull,
      args = list(shape = 1.5, scale = 1),
      xlim = c(0, 4),
      color = "black",
      linewidth = 1
    ) +
    scale_color_manual(
      values = c(
        "above" = "firebrick",
        "on/below" = "forestgreen"
      )
    ) +
    coord_cartesian(xlim = c(0, 4), ylim = c(0, 0.8)) +
    labs(
      title = paste("n =", n),
      subtitle = paste(
        "Est. Numerical Integration:",
        round(result$estimate, 4)
      ),
      x = "x",
      y = "y",
      color = "flag"
    ) +
    theme_minimal() +
    theme(legend.position = "bottom")
}

# Combine four resolutions into the small-multiple display.
plot10 <- build_mc_plot(10)
plot100 <- build_mc_plot(100)
plot1000 <- build_mc_plot(1000)
plot10000 <- build_mc_plot(10000)

mc_small_multiple <- (plot10 + plot100) / (plot1000 + plot10000)
```